K8sGPT resolve 70% dos seus problemas.

Vou te mostrar como resolver os outros 30% com um agente local

Emerson Silva

TDC São Paulo



4·Linux Open Software Specialists ™

Emerson Silva



- Engenheiro DevOps na **4Linux**
- +9 anos em ambientes DevOps críticos
- Foco em **Kubernetes**, IaC e confiabilidade
- Palestrante, instrutor,escritor , ex-zagueiro
- Community Lead de Kubernetes na DOUGBrazil
- Organizador do CNCF Campinas

CLOUD NATIVE COMPUTING FOUNDATION CAMPINAS

DOUG DEVOPS USER GROUP BRAZIL

4·Linux Open Software Specialists ™

4·Linux Open Software Specialists™

Fundada em 2001, a 4Linux participou das principais transformações da área de tecnologia. Começamos com Linux e software livre, navegamos por Cloud, DevOps e Containers — e hoje atuamos com agentes de IA, sempre com base em **inovação aberta**.

2001
Linux & Software Livre

2008
DevOps

2016
Containers & Kubernetes

2019
Machine Learning

2020
Cloud

2024+
IA com Open Source

Agenda

1. O que é o K8sGPT — e onde ele já acerta
2. O que falta, mesmo assim
3. O que é o k8s-mentor, e por que 100% local
4. Os 5 pilares — um print real por vez
5. Fase 6: rodando sozinho, avisando no Slack
6. Testado de verdade, e quem decide no final

O K8SGPT

O que é o K8sGPT

Projeto **CNCF Sandbox** (aceito em dez/2023), Apache-2.0, escrito em Go.

- **8,2k stars** e **1,0k forks** no GitHub
- **14 analyzers** habilitados por padrão (Pod, Deployment, Service, Ingress, PVC, Node...) + 21 opcionais
- **15 backends de IA** suportados — OpenAI, Azure, Bedrock, Cohere... e local: **Ollama**, LocalAI
- Escaneia o cluster, roda os analyzers, manda o problema pro LLM traduzir em explicação legível

Não é hype — é um projeto CNCF maduro, mantido ativamente, adotado de verdade. A base desta talk.

O PROBLEMA

O que falta no K8sGPT

- **Stateless** — cada análise começa do zero
- Não sabe se **já aconteceu antes**, nem quantas vezes
- Não conhece o **runbook** que seu time escreveu
- Não correlaciona com **deploys recentes**
- Não vê **tendência** de métrica antes do crash

Ele responde "o que quebrou". O humano ainda cata o "por quê" em 3 painéis — e isso consome horas de plantão que não precisavam ser gastas assim.

A SOLUÇÃO

k8s-mentor

Roda **do lado** do K8sGPT (não substitui) e adiciona:

1. Memória de incidentes
2. RAG sobre runbooks internos
3. Correlação com deploys (GitOps)
4. Correlação com métricas (Prometheus)
5. Ação supervisionada — nunca decide sozinho

100% local: Ollama + Qdrant. Sem depender de API externa, sem expor dado sensível nem secret do cluster pra fora da rede.

100% LOCAL

Por que isso importa de verdade

- **Dado sensível não sai da rede** — nome de recurso, mensagem de erro, runbook interno, secret mal higienizado num log: nada disso vira payload pra API de terceiro
- **Sem custo por token** — rodando sozinho a cada 5min, uma API paga vira conta que ninguém previu
- **Sem depender de internet no meio do incidente** — o cluster caiu, sua saída pra nuvem também pode ter caído junto
- **Sem vendor lock-in** — troca de modelo é `ollama pull`, não migração de contrato

Ollama rodando **Qwen2.5**, Qdrant pra memória vetorial — os dois dentro do mesmo cluster ou no seu laptop, sem GPU dedicada

ESCOLHA DE MODELO

Por que Qwen2.5

- **Roda sem GPU dedicada** — variantes pequenas cabem no laptop de demo
- **Segue formato de saída de forma consistente** — o pipeline inteiro depende de JSON bem formado voltando do modelo
- **Apache 2.0** — mesma licença do K8sGPT, stack open source de ponta a ponta

POR QUE "MENTOR"?

O nome não é acaso

Na *Odisseia*, **Méntor** é o amigo a quem Ulisses confia seu filho Telêmaco antes de ir pra guerra — não pra decidir por ele, mas pra aconselhar.

Um mentor não aperta o botão por você. Ele **conhece seu histórico**, **já viu o problema antes**, **aponta o caminho** — a decisão final continua sua.

É exatamente o papel deste agente ao lado do K8sGPT.



MOMENTO 1

K8sGPT sozinho

```
emerson@odin ~/k8s-mentor k8sgpt analyze --namespace demo --backend ollama --explain -o json \
 --kubecontext kind-k8s-mentor-demo | python3 -m json.tool

W0908 15:41:36.013181  292436 warnings.go:70] v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
{
    "provider": "ollama",
    "errors": null,
    "status": "ProblemDetected",
    "problems": 3,
    "results": [
        {
            "kind": "ConfigMap",
            "name": "demo/kube-root-ca.crt",
            "error": [
                {
                    "Text": "ConfigMap kube-root-ca.crt is not used by any pods in the namespace",
                    "KubernetesDoc": "",
                    "Sensitive": []
                }
            ],
            "details": "\nError: The ConfigMap kube-root-ca.crt is not used by any pods in the namespace.\n\nSolution:\n1. Check if the ConfigMap i
s correctly defined and accessible in the namespace.\n2. Ensure that the pods are using the correct CA certificate by checking the pod's configurat
ion files or Kubernetes environment variables.\n3. If issue persists, check if there are any networking issues between the pods and the ConfigMap."
,
            "parentObject": ""
        },
        {
            "kind": "Deployment",
            "name": "demo/image-resizer",
            "error": [
                {
```

O que falta no K8sGPT
cada análise começa do zero
Não sabe se já aconteceu antes, nem quantas vezes
o runbook que seu time escreveu
Não correlaciona com deploys recentes
responde "o que quebrou". O humano ainda caça o "por que" em 3 painéis.

Quem faz o quê

K8sGPT

- Escaneia o cluster em tempo real
- Traduz erro críptico em explicação legível
- Cobre Pods, Deployments, PVCs, RBAC...
- Cada análise começa do zero

k8s-mentor

- Lembra do que já aconteceu antes
- Cruza com o runbook que seu time escreveu
- Aponta o deploy que provavelmente causou
- Confirma a tendência real na métrica

Consome a saída do K8sGPT — não substitui, adiciona o contexto que falta.

MOMENTO 2

Memória + RAG + Métricas


```
{
    "score": 1.0000002,
    "kind": "Deployment",
    "name": "checkout-service",
    "namespace": "demo",
    "message": "Deployment demo/checkout-service has 1 replicas but 0 are available with status running",
    "timestamp": "2026-09-08T18:40:34Z"
},
{
    "score": 0.9843444,
    "kind": "Deployment",
    "name": "checkout-service",
    "namespace": "demo",
    "message": "Deployment demo/checkout-service has 1 replicas but 0 are available with status running",
    "timestamp": "2026-09-08T17:50:44Z"
},
{
    "score": 0.9831348,
    "kind": "Deployment",
    "name": "checkout-service",
    "namespace": "demo",
    "message": "Deployment demo/checkout-service has 1 replicas but 0 are available with status running",
    "timestamp": "2026-09-08T17:35:45Z"
},
{
    "score": 0.9793547,
    "kind": "Deployment",
    "name": "checkout-service",
    "namespace": "demo",
    "message": "Deployment demo/checkout-service has 1 replicas but 0 are available with status running",
    "timestamp": "2026-09-08T17:55:51Z"
},
```

Mesmo erro, contexto real

- `recurrence: 9` — similaridade **semântica**, não palavra-chave
- Runbook certo achado: `checkout-service-memory-leak.md` (score 0.72)
- Prometheus confirma: memória em **crescimento sustentado**

MOMENTO 3

Ação supervisionada


```
**Check logs:** Inspect the container logs for specific errors. Use `kubectl logs <pod_name>`.\n2. **Inspect Pod Status:** Check the pod status us
ng `kubectl get pods <pod_name>`.  \n3. **Resource Constraints:** Ensure your pod has enough resources (CPU, memory) and network capacity. \n4. **
mage Pull:** Verify the container image is available and pulling correctly. Use `kubectl describe pod <pod_name>` to check for image issues.\n5. *
Network Connectivity:** Check if the pod can reach necessary services or networks.  \n\n\n---",
                "timestamp": "2026-09-08T18:46:03.378897664Z"
            },
            "history": {
                "recurrence": 0,
                "similar_incidents": null
            },
            "gitops": {
                "correlated": true,
                "suspect_recent_deploy": true,
                "deployment": "payment-worker",
                "replica_set": "payment-worker-647b88449f",
                "revision": "2",
                "commit": "e4f5a6b \u2014 fix: normaliza nome das env vars do payment-worker",
                "deployed_at": "2026-09-08T15:44:35-03:00",
                "gap_seconds": 88.378897664,
                "previous_replica_set": "payment-worker-6ddbd6b6c4",
                "previous_revision": "1"
            },
            "action": {
                "type": "rollback",
                "reason": "deploy recente (revis\u00e3o 2, 88s antes do erro) \u00e9 a causa mais prov\u00e1vel \u2014 voltar pra revis\u00e3o 1",
                "namespace": "demo",
                "deployment": "payment-worker",
                "replica_set": "payment-worker-6ddbd6b6c4",
                "revision": "1",
                "equivalent_command": "kubectl rollout undo deployment/payment-worker -n demo --to-revision=1"
            }
        }
    ]
}
```

Sugerir → aprovar → confirmar

```
 emerson@odin  ~/k8s-mentor  jq '.incidents[0].action' /tmp/pw-analyzed.json > /tmp/suggestion.json

 emerson@odin  ~/k8s-mentor  K8S_MENTOR_KUBE_CONTEXT=kind-k8s-mentor-demo go run ./cmd/k8s-mentor apply-action --input /tmp/suggestion.json

Ação sugerida: rollback
Motivo: deploy recente (revisão 2, 88s antes do erro) é a causa mais provável — voltar pra revisão 1
Comando equivalente: kubectl rollout undo deployment/payment-worker -n demo --to-revision=1

Nada foi executado. Rode de novo com --yes pra aplicar.
 emerson@odin  ~/k8s-mentor
```

```
 emerson@odin  ~/k8s-mentor  K8S_MENTOR_KUBE_CONTEXT=kind-k8s-mentor-demo go run ./cmd/k8s-mentor apply-action --input /tmp/suggestion.json --

Ação sugerida: rollback
Motivo: deploy recente (revisão 2, 88s antes do erro) é a causa mais provável — voltar pra revisão 1
Comando equivalente: kubectl rollout undo deployment/payment-worker -n demo --to-revision=1
✅ Ação aplicada.
 emerson@odin  ~/k8s-mentor
```

```
4: emerson@odin:~/k8s-mentor
 emerson@odin  ~/k8s-mentor  kubectl get pods -n demo -l app=payment-worker
kubectl logs -n demo -l app=payment-worker --tail=3

NAME                              READY   STATUS        RESTARTS   AGE
payment-worker-5667555859-t2skc   1/1     Running       0          19s
payment-worker-6ddbd6b6c4-hs5c4   1/1     Terminating   0          3m43s
payment-worker conectado a postgres://payments-db:5432/payments
payment-worker conectado a postgres://payments-db:5432/payments
 emerson@odin  ~/k8s-mentor
```

FASE 6

Rodando sozinho, avisando no Slack

Até aqui, tudo rodou porque eu digitei um comando. Um `CronJob` dentro do próprio cluster roda o k8sgpt sozinho, passa pelos mesmos pilares e — quando acha algo — notifica um canal do Slack.

Repara o que a mensagem **não** faz: não diz "eu resolvi".
Diz o que sugere, o porquê, e o comando pra aprovar.

E notifica **com critério**: só interrompe o canal quando o padrão se confirma — recorrência, tendência de métrica ou ação sugerida. Evento isolado fica só no log.

Hoje

Novo

k8s-mentor APP 15h49

🔍 **k8s-mentor** encontrou **5** incidente(s) em `demo`

- **Deployment/checkout-service**
  Deployment demo/checkout-service has 1 replicas but 0 are available with status running
  recorrência: 18x nos últimos registros

- **Deployment/image-resizer**
  Deployment demo/image-resizer has 1 replicas but 0 are available with status running
  recorrência: 19x nos últimos registros

- **ConfigMap/kube-root-ca.crt**
  ConfigMap kube-root-ca.crt is not used by any pods in the namespace
  recorrência: 11x nos últimos registros

- **Pod/checkout-service-6f6c98585b-sw24r**
  the last termination reason is OOMKilled container=checkout-service pod=checkout-service-6f6c98585b-sw24r
  recorrência: 20x nos últimos registros
  💡 runbook: `demo/runbooks/checkout-service-memory-leak.md`
  ⚠️ **ação sugerida: restart** — memória em crescimento sustentado (padrão de vazamento) — reiniciar é só paliativo, não corrige a causa raiz; ver runbook (--with-runbooks) pra correção de verdade
  aprove com:

```
k8s-mentor apply-action --input <arquivo-com-a-sugestao> --yes
```

- **Pod/image-resizer-75f859f6c7-qwzqs**
  the last termination reason is OOMKilled container=image-resizer pod=image-resizer-75f859f6c7-qwzqs
  recorrência: 20x nos últimos registros

Nada foi executado automaticamente — nenhuma ação acontece sem aprovação manual explícita.

```
Deployment/checkout-service: Deployment demo/checkout-service has 1 replicas but 0 are available with status running [recorrência: 18x]
Deployment/image-resizer: Deployment demo/image-resizer has 1 replicas but 0 are available with status running [recorrência: 19x]
ConfigMap/kube-root-ca.crt: ConfigMap kube-root-ca.crt is not used by any pods in the namespace [recorrência: 11x]
Pod/checkout-service-6f6c98585b-sw24r: the last termination reason is OOMKilled container=checkout-service pod=checkout-service-6f6c98585b-sw24r [recorrência: 20x] [runbook:
demo/runbooks/checkout-service-memory-leak.md] [tendência: memória em crescimento] [AÇÃO SUGERIDA: restart]
Pod/image-resizer-75f859f6c7-qwzqs: the last termination reason is OOMKilled container=image-resizer pod=image-resizer-75f859f6c7-qwzqs [recorrência: 20x]
```

CREDIBILIDADE

Isso é testado de verdade

Cada pilar foi validado contra um cluster **real** e a saída **real** do K8sGPT — não JSON forjado à mão:

- Parser ajustado ao formato real de saída do K8sGPT
- Thresholds de similaridade calibrados com dados reais, não texto de exemplo
- Detecção de tendência endurecida no caso real de vazamento de memória
- Correlação de métrica corrigida pra pegar a série certa no Prometheus
- Notificação do Slack com filtro de recorrência, pra avisar só o que importa

Não é slideware. É a diferença entre uma demo que parece funcionar e uma que funciona.

Stack técnica

Componente	Escolha
Agente	**Go** (ecossistema K8s nativo)
LLM local	**Ollama** + **Qwen2.5**
Memória vetorial	**Qdrant**
Cluster de demo	**Kind**, 3 nodes
Diagnóstico (fonte)	**K8sGPT**, desacoplado

Roadmap

- ✅ 5 pilares + Fase 6 (execução periódica + Slack) — **prontos e testados**
- 

Síntese via LLM (prosa em vez de JSON)
- 

Alertmanager de verdade (evento, não só polling por tempo)

Quem decide?

Quanto mais forte o sinal — mais recorrência, mais correlação — mais fácil é a tentação de deixar a máquina decidir sozinha.

k8s-mentor foi desenhado pra resistir essa tentação.

Ele fica mais inteligente. A decisão continua sendo sua.

Curso gratuito

DevOps, AIOps e SRE Fundamentals

Curso gratuito da 4Linux

Escaneie o QR code pra se inscrever

- 🎓 4linux.com.br/cursos



4·Linux Open Software Specialists™

Obrigado!

Contato

- 🔗 github.com/silvemerson/k8s-mentor
- 📝 emerson-silva.blog.br
- 💼 linkedin.com/in/silvemerson

Vamos manter contato!

